

ISSU — Project Blueprint

Project Codename: ISSU

Blueprint Version: 0.1

Project Status: Pre-Development / Foundation

License: Apache License 2.0

Project Type: Open-Source Autonomous Intelligence Platform

1. Project Identity

ISSU is an open-source project focused on building a modular, autonomous intelligence platform capable of performing complex tasks across multiple domains.

The project will initially focus on autonomous software engineering as its first major application domain.

The long-term objective is to develop a general-purpose foundation that can be extended into many fields through independent capabilities, tools, agents, and domain modules.

The name **ISSU** is a temporary project codename and may be replaced by a permanent name in the future.

2. Vision

Our vision is to build an open-source autonomous intelligence platform capable of understanding objectives, planning work, using tools, executing actions, evaluating results, learning from context, and completing complex tasks with increasing levels of autonomy.

The platform should not remain limited to coding.

Software engineering is our starting point and proving ground.

The long-term platform should be capable of supporting applications across fields such as:

- Software engineering
- Research
- Education
- Business automation
- Robotics
- Science
- Engineering
- Data analysis
- Creative workflows

- Other future domains

The architecture must therefore be designed for expansion from the beginning.

3. Mission

Our mission is to build a professional-grade, open-source foundation for autonomous AI systems that developers and communities around the world can use, extend, study, and improve.

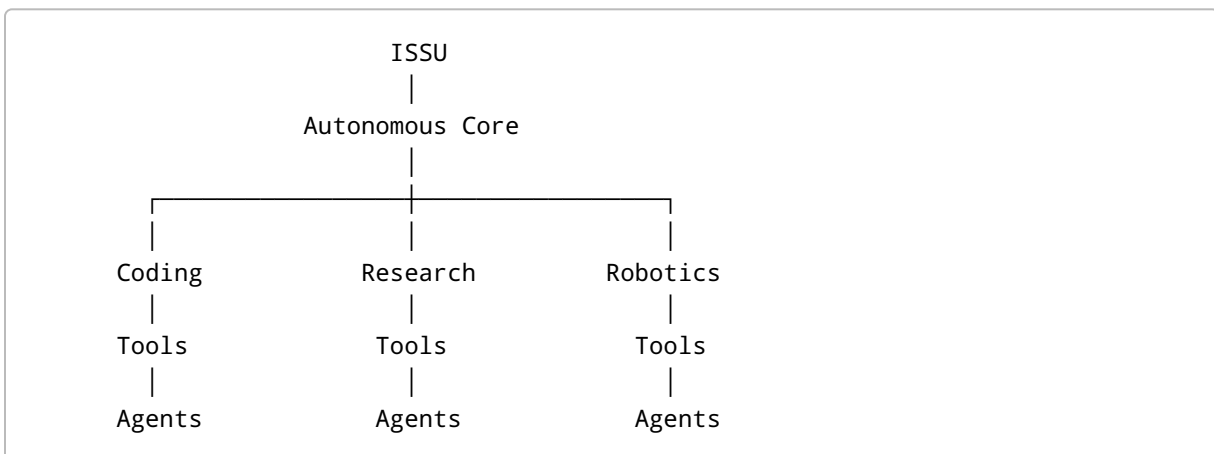
ISSU should make it possible to construct increasingly capable autonomous systems without requiring the entire platform to be redesigned for every new domain.

4. Long-Term Objective

ISSU aims to become more than an AI coding agent.

The long-term objective is to create an extensible autonomous intelligence ecosystem in which a common core provides fundamental capabilities while independent modules provide specialized domain capabilities.

Conceptually:



The same underlying platform should be capable of supporting entirely different applications without requiring fundamental changes to the core.

5. Initial Scope

The first concrete objective is to build a powerful autonomous software engineering agent.

The initial system should progressively acquire the ability to:

- Understand a software project
- Understand user objectives
- Plan multi-step tasks
- Read and modify files
- Execute commands
- Use development tools
- Run tests
- Diagnose failures
- Correct errors
- Review its own work
- Maintain useful context
- Work autonomously within defined boundaries
- Interact with Git and other development systems
- Research information when necessary

The first domain is deliberately chosen because software engineering provides an excellent environment for testing autonomous reasoning, planning, tool usage, verification, and iterative problem solving.

6. Future Scope

After establishing a strong autonomous foundation, ISSU may expand into additional domains.

Potential future domain modules include:

- Research agents
- Education agents
- Business agents
- Scientific agents
- Robotics agents
- Data and analytics agents
- Engineering agents
- Creative agents
- Personal productivity agents
- Specialized industry agents

These domains should be added through modular extensions rather than by turning the core into a collection of unrelated features.

7. Core Engineering Principles

7.1 Build Foundations, Not Features

We prioritize strong foundations over rapidly accumulating features.

Every feature should contribute to a sustainable architecture.

7.2 Modularity First

Components should have clear responsibilities and well-defined interfaces.

A component should be replaceable without requiring unnecessary changes throughout the system.

7.3 Independent Development

Each development phase should be independently developed, tested, documented, and understood.

A phase should not depend on another phase's internal implementation.

7.4 Interface-Based Integration

Phases and modules may communicate with one another, but communication should occur through explicitly defined interfaces, contracts, protocols, or adapters.

Internal implementation details must remain isolated.

7.5 Documentation is Part of the Product

Documentation is not an optional addition.

A phase is incomplete if its architecture, interfaces, behavior, limitations, and usage are not documented.

7.6 Performance by Design

Performance should be considered during architecture and implementation rather than treated as an afterthought.

Efficiency should be measured where appropriate.

7.7 Reliability Over Unnecessary Complexity

The simplest architecture capable of solving the problem should generally be preferred.

Complexity must have a clear justification.

7.8 Security by Default

The platform will eventually have access to powerful tools such as terminals, filesystems, networks, credentials, and external services.

Security therefore must be considered from the earliest architectural stages.

7.9 Extensibility

The platform should make it possible to add capabilities without repeatedly modifying the fundamental core.

7.10 Open-Source Quality

Code should be written as though other developers will eventually inspect, use, modify, and contribute to it.

7.11 Learn While Building

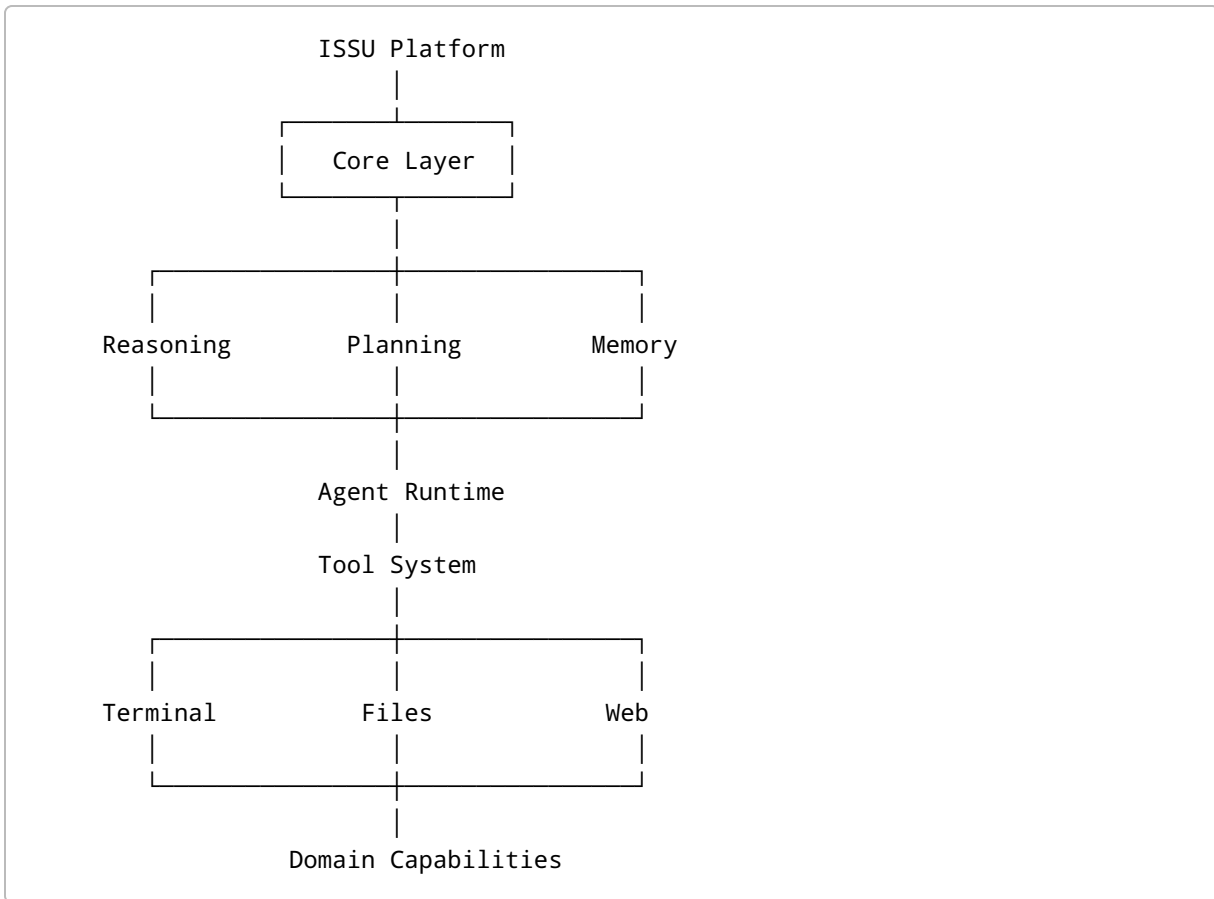
The project is simultaneously a serious engineering effort and a learning journey.

Architectural decisions should be explained and understood rather than blindly implemented.

8. Architecture Philosophy

ISSU will follow a modular architecture.

At a high level, the platform is expected to evolve toward components such as:



This is a conceptual architecture, not the final implementation.

The actual architecture will be refined during the relevant development phases.

9. Phase Architecture

ISSU will be developed through multiple phases.

Each phase will have its own dedicated folder and development lifecycle.

A phase should contain, where appropriate:

```
README.md
ARCHITECTURE.md
SPECIFICATION.md
TASKS.md
src/
```

```
tests/  
examples/
```

Not every phase must contain exactly the same files. Structure should serve the phase rather than become unnecessary bureaucracy.

10. Phase Independence

Phase independence is one of the project's most important architectural goals.

Each phase must:

- Have a clearly defined responsibility.
- Have its own implementation.
- Have its own tests.
- Have its own documentation.
- Define its public interface.
- Avoid depending on another phase's internal files.
- Be understandable independently.
- Be replaceable where practical.

The final system will combine the phases through defined integration boundaries.

Important distinction

Independence does **not** mean that phases can never communicate.

It means:

A phase depends on another phase's contract, not its implementation.

11. Development Lifecycle

Every phase follows a disciplined lifecycle:

```
Define  
↓  
Research  
↓  
Architect  
↓  
Specify
```

```
graph TD; A[ ] --> B[Implement]; B --> C[Test]; C --> D[Review]; D --> E[Refactor]; E --> F[Document]; F --> G[Freeze]; G --> H[Next Phase];
```

↓
Implement
↓
Test
↓
Review
↓
Refactor
↓
Document
↓
Freeze
↓
Next Phase

A phase should not be considered complete merely because its code runs.

12. Testing Philosophy

Testing will be treated as a fundamental engineering requirement.

Depending on the phase, testing may include:

- Unit testing
- Integration testing
- Functional testing
- Failure testing
- Security testing
- Performance testing
- Regression testing
- End-to-end testing

Autonomous systems require particular attention to failure handling because successful execution cannot be assumed.

13. Autonomous Agent Philosophy

ISSU should eventually operate around a controlled agent loop.

Conceptually:


```
Goal
↓
Understand
↓
Plan
↓
Select Action
↓
Use Tool
↓
Observe Result
↓
Evaluate
↓
Correct / Continue
↓
Verify
↓
Complete
```

The exact implementation will be determined during the appropriate phases.

The agent must not simply generate actions.

It should progressively become capable of **evaluating whether those actions achieved the intended result**.

14. Tool Philosophy

Tools should be treated as modular capabilities rather than hard-coded features.

Potential tools include:

- Filesystem
- Terminal
- Git
- Web search
- Browser
- Documentation retrieval
- Database systems
- APIs
- Testing frameworks
- Deployment systems

- External services

New tools should be addable without redesigning the entire agent.

15. Memory Philosophy

Memory should eventually be treated as a modular subsystem.

Different forms of memory may be required, including:

- Current task context
- Conversation context
- Project context
- Long-term information
- Learned preferences
- Tool history
- Important decisions

The actual memory architecture will be designed and evaluated during the relevant phase.

16. Multi-Agent Philosophy

ISSU may eventually support multiple specialized agents.

Potential roles could include:

```
Manager
Architect
Researcher
Developer
Tester
Reviewer
Security Agent
Documentation Agent
```

However, multi-agent architecture should only be introduced where it provides measurable value.

We will not add multi-agent complexity merely because it sounds impressive.

17. Security Philosophy

Autonomous systems can potentially execute powerful actions.

Therefore, ISSU must eventually consider:

- Permission boundaries
- Sandboxing
- Command restrictions
- Filesystem permissions
- Credential protection
- Network access
- Tool authorization
- User confirmation
- Audit logs
- Safe failure modes
- Malicious input
- Prompt injection
- Untrusted repositories

Security architecture will be developed progressively rather than postponed until the final stage.

18. Model Independence

ISSU should avoid unnecessary dependence on a single AI model provider.

Where practical, the architecture should support multiple model backends.

Potential model categories may include:

- Cloud models
- Local models
- Open-source models
- Specialized models
- Future model architectures

The exact providers and APIs will be decided during implementation.

19. Technology Decision Philosophy

Technology choices should be based on:

- Reliability
- Performance
- Maintainability
- Ecosystem maturity
- Developer experience
- Security
- Extensibility
- Community support
- Long-term viability

We will not choose technologies merely because they are currently popular.

20. Open-Source Strategy

ISSU will be developed as an open-source project.

The project should eventually provide:

- Clear documentation
- Contribution guidelines
- Issue tracking
- Development guidelines
- Architecture documentation
- Changelogs
- Versioned releases
- Community contribution mechanisms

The project should be approachable for new contributors while maintaining professional engineering standards.

21. License

ISSU will use:

Apache License 2.0

The license provides broad freedom to use, modify, distribute, and build upon the project while providing explicit patent-related protections.

The repository will contain the official `LICENSE` file at its root.

22. Versioning

ISSU will use semantic versioning where appropriate.

Conceptually:

```
0.x.x → Development
1.0.0 → First stable release
1.x.x → Stable feature development
2.x.x → Major architectural evolution
```

Major version changes should represent meaningful compatibility or architectural changes.

23. Configuration Philosophy

Configuration should be centralized, understandable, and extensible.

Users should eventually be able to configure things such as:

- Models
- Providers
- Tools
- Permissions
- Memory
- Agent behavior
- Project settings
- Logging
- Performance settings

Configuration mechanisms will be established during the relevant phase.

24. Observability

An autonomous system must be understandable when something goes wrong.

ISSU should eventually provide appropriate observability mechanisms such as:

- Logs
- Agent activity
- Tool calls
- Errors
- Decisions
- Task progress
- Performance metrics
- Debugging information

Observability should help developers understand **what the agent did and why**.

25. Integration Philosophy

Integration will occur only after individual components have reached sufficient stability.

The final integration stage will:

1. Identify completed phase interfaces.
 2. Build adapters where necessary.
 3. Connect modules.
 4. Run integration tests.
 5. Run end-to-end tests.
 6. Identify architectural conflicts.
 7. Refactor where necessary.
 8. Validate the complete system.
 9. Prepare the first complete release.
-

26. Non-Goals

ISSU will not initially attempt to:

- Solve artificial general intelligence.
- Replace every existing AI platform.
- Support every possible domain immediately.
- Build every feature simultaneously.
- Optimize for marketing before engineering quality.
- Copy another AI coding agent feature-for-feature.
- Lock the architecture to a single model provider.

The long-term vision can be extremely ambitious while the implementation remains disciplined and incremental.

27. Product Philosophy

ISSU should not simply imitate existing systems.

Existing autonomous coding systems will be studied as sources of engineering knowledge and inspiration, but ISSU should develop its own architecture and ideas.

The goal is:

Learn from what exists. Build what is missing. Improve what can be improved.

Innovation should be driven by actual engineering problems and user needs.

28. Quality Standard

Every major component should eventually answer:

- What problem does it solve?
- Why does it exist?
- What is its interface?
- What are its inputs?
- What are its outputs?
- What can fail?
- How is failure handled?
- How is it tested?
- How is it documented?
- Can it be replaced?
- Does it introduce unnecessary complexity?

If these questions cannot be answered, the component is not sufficiently understood.

29. Decision-Making Principle

When multiple technical approaches are possible, decisions should be evaluated using:

Correctness

↓

Security

↓

Maintainability

↓
Performance
↓
Extensibility
↓
Developer Experience
↓
Complexity

The exact ordering may change for specific situations, but correctness and security should never be sacrificed merely for convenience.

30. Governance

During the initial development period:

Founder / Lead Developer: Project Owner

Architecture: Guided by the Chief System Architect

Major architectural decisions should be documented rather than existing only in conversation.

As the open-source community grows, governance can evolve.

31. Project Roles

Founder / Lead Developer

Responsible for:

- Project direction
- Implementation
- Repository management
- Product decisions
- Community direction

Chief System Architect

Responsible for:

- Overall architecture
- System boundaries

- Architectural decisions
- Technical consistency

AI Architect

Responsible for:

- Agent architecture
- Reasoning systems
- Planning
- Memory
- Tool orchestration
- Multi-agent systems

Technical Mentor

Responsible for:

- Explaining technical concepts
- Teaching implementation decisions
- Helping develop engineering skills

Code Reviewer

Responsible for:

- Reviewing implementation
- Identifying defects
- Maintaining quality

Project Strategist

Responsible for:

- Long-term direction
- Prioritization
- Preventing unnecessary scope expansion

These roles may initially be fulfilled by the project founder and AI assistance, while remaining conceptually separated.

32. Development Environment

The project will initially be developed using an AI coding agent such as OpenCode as a development assistant.

OpenCode is a development tool used to build ISSU.

It is **not** ISSU itself.

The architecture of ISSU must remain independent of the development tool used to create it.

33. Development Discipline

We will follow these rules:

1. Do not skip phases without justification.
 2. Do not implement features without understanding their purpose.
 3. Do not blindly accept AI-generated code.
 4. Review generated code.
 5. Test implementations.
 6. Document important decisions.
 7. Keep modules isolated.
 8. Avoid unnecessary dependencies.
 9. Avoid premature optimization.
 10. Avoid premature abstraction.
 11. Do not allow temporary hacks to silently become permanent architecture.
 12. Prefer measurable engineering decisions.
-

34. Project Motto

Build once. Extend forever.

This represents the central philosophy of ISSU:

Build strong foundations that can support capabilities far beyond the first application.

35. Current Status

VISION	✓
MISSION	✓
PROJECT SCOPE	✓
LONG-TERM DIRECTION	✓
ROLES	✓
ENGINEERING PRINCIPLES	✓
MODULAR STRATEGY	✓

PHASE STRATEGY	✓
OPEN-SOURCE DIRECTION	✓
LICENSE	✓
BLUEPRINT	✓
IMPLEMENTATION	⌚

36. Next Step

The planning stage is now considered complete.

The project will now enter:

PHASE 1 — FOUNDATION

Phase 1 will establish the technical foundation of ISSU.

No later phase should be implemented before its architectural requirements are understood.

The exact Phase 1 architecture, folder structure, specifications, tooling, and implementation tasks will be defined at the beginning of Phase 1.

Final Principle

ISSU is being built with ambition, but ambition will never replace engineering discipline.

We will move quickly where possible, but we will not sacrifice the foundations that make future development possible.

The objective is not simply to build an AI agent.

The objective is to build a foundation upon which increasingly capable autonomous systems can be built.